DATA-303

Applied Machine Learning, Bias & Ethics
HW 1: Introduction to Machine Learning & Linear Regression

Name: Raven Warner
Date: 2/7/2026

Instructions

- Provide short written answers (PDF/Word/Markdown) for conceptual questions, plus any code (Jupyter notebook, .py file, or matlab files) for implementation tasks.
- Show your work, including how you compute metrics or arrive at numeric answers.
- You may discuss ideas with peers, but your final submission must be your own.

Q1: Basic Concepts in Machine Learning (15 points)

1. Types of Learning (8 points total)

a) (4 pts) In your own words, describe the difference between supervised and unsupervised learning.

YOUR ANSWER:

- Supervised Learning
    - I would describe supervised learning as learning that a computer, algorithm, machine does by ingesting labeled data. The data is labeled distinctly, for example, it could be vehicle data where the goal is to determine whether the vehicle is a truck or a car. The data would consist of pictures labeled with truck or labeled with car depending on the vehicle type.
- Unsupervised Learning
    - Referencing the response above for additional context, unsupervised learning would be when the machine, computer, or specifically an algorithm takes in unlabeled data and discovers patterns or groupings on its own, though it may not know exactly what they are (like a car or truck), but make the determination that cars and trucks are not the same. For example, instead of providing labeled cars and labeled trucks, I could simply provide a bunch of pictures of cars and trucks with no labels to see if the distinction between a car and a truck could be made on its own.

b) (4 pts) What is the main difference between a classification task and a regression task? Provide an example of each.

YOUR ANSWER:

- **Classification** task - This predicts what data goes into classes or groupings. For example, whether an email is a fraudulent (i.e., phishing, whaling, etc.) email or normal communications.
- **Regression** - This predicts a numerical outcome based on previously provided data, like predicting actual rent prices based on location, square footage, and amenities.

2. Concepts & Strategy (7 points)

a) The Hierarchy (4 pts): In a job interview, a recruiter asks: "Is Deep Learning a separate field from Machine Learning, or is it related?"

Answer this question by describing the relationship between Artificial Intelligence, Machine Learning, and Deep Learning.

YOUR ANSWER:

- Artificial Intelligence is the discipline that seeks to dive into machines mimicking or replicating human intelligence.
- Machine learning is a subset of artificial intelligence where it focuses on the studies and methods behind how machines, computers, and systems learn.
- Deep learning is a subset of machine learning, that specifically focuses on learning through neural networks.

Can you have an AI system that is not Machine Learning? Give one simple example.

- Yes. A good example could be the older/early GPS/Garmin systems that leverage best path/route algorithms, but do not learn from your driving (or the data associated with your driving).

b) When NOT to use ML (3 pts): A Senior Data Scientist asks you: "Give me an example of a problem where you would refuse to use Machine Learning."

- Referencing the "Lion vs. Deer" scenario (simple vs. complex problems), explain why we generally don't use ML for problems with clear, deterministic rules (like converting Celsius to Fahrenheit or calculating a fixed tax rate).

YOUR ANSWER:

- Some things do not require machine learning to complete, and I have always been taught to leverage what has already been done and not to reinvent the wheel. For example, I do not need to implement a machine learning to conduct simple math like 2+2 or 4*5. A calculator is a perfect use case for solving a problem without machine learning. In the "Lion vs. Deer" scenario, ML is like using a lion to catch a deer when you could just use a simple trap (deterministic formula). For temperature conversion, we have a perfect mathematical formula that works 100% of the time with limited error (i.e., user error).

Q2: Linear Regression Fundamentals (30 points)

1. The Cost Function (8 points)

a) (4 pts) In Linear Regression, we minimize the Sum of Squared Errors (SSE): Error = $(y - \hat{y})^2$. Why do we square the error instead of just taking the absolute difference?

YOUR ANSWER:

- We square the error because it removes the possibility of negative error values and emphasizes larger discrepancies of errors in the model, penalized those errors more heavily.

b) (4 pts) If your trained model consistently predicts values higher than the actual values, what does that suggest about your intercept term (θ_0)?

YOUR ANSWER:

- If a trained model often predicts values higher than the actual values, it suggests that the intercept term is too large. A large intercept shifts all predictions upward, leading to systematic overprediction. Reducing during training would help align the predictions more closely with the true values.

2. Gradient Descent & The Update Rule (10 points)

In class, we looked at the update rule for the weights in Linear Regression model:
$w_{n+1} = w_n - \eta \cdot (2 \cdot \text{Error}) \cdot x_n$

a) (4 pts) Explain the role of η (learning rate). What happens if it is too huge?

YOUR ANSWER:

- The learning rate is a small positive number (e.g., 0.01) that controls how large of a step we take. If α is too large, we might overshoot the bottom entirely and fail to find the minimum. Meanwhile, if it is too small, training the model will take a very long time, and several iterations.

b) (6 pts) Explain conceptually why we subtract the error term. If the Error is positive (Prediction is too high), how does this formula adjust the weight?

YOUR ANSWER:

- We subtract the gradient because it points in the direction to where the error increases. By reducing the error, we move in the opposite direction, which is why we subtract instead of adding. This moves the model toward lower error. When the prediction is too high, the error is positive. When the weight decreases, future predictions will become smaller to help correct the over-prediction.

3. Manual Gradient Descent Mini Example (12 points)

Suppose you have the following tiny dataset of two points:
$(x_1, y_1) = (1, 2)$
$(x_2, y_2) = (2, 3)$

(8 pts) Initialize $\theta_0 = 0$ and $\theta_1 = 0$. Let your learning rate $\alpha = 0.1$. Perform two iterations of gradient descent by hand (or in a spreadsheet). Show:

- The predicted values at each iteration
- The error terms
- The updated values of θ_0 and θ_1

YOUR ANSWER:
Iteration 1
$\theta_0 = 0$
$\theta_1 = 0$
$\alpha = 0.1$
Point 1: $(x_1, y_1) = (1, 2)$
Point 2: $(x_2, y_2) = (2, 3)$

Prediction formula: $\hat{y} = \theta_0 + \theta_1 \cdot x$

$\hat{y}_1 = 0+0 * 1 = 0$
$\hat{y}_2 = 0+0 * 2 = 0$

Error formula: $\hat{y} - y$ (predicted – actual)
Error1 for Point1: $(\hat{y}_1 = 0, y_1 = 2)$ 0-2=-2
Error2 for Point2: $(\hat{y}_2 = 0, y_2 = 3)$ 0-3 = -3

Update formula (intercept) is: **θ_0_new = θ_0_old - α · (average of errors)**

θ_0_new = **0-0.1((-2+-3)/2))**

θ_0_new = **0-0.1(-2.5)**

θ_0_new = **0.25**

Update formula (slope) is: **θ_1_new = θ_1_old - α · (average of errors × x)**

θ_1_new = θ_1_old - α · (average of [Error × x])

$Error_1 \times x_1$: $(Error_1 = -2, x_1 = 1)$ -2*1=-2
$Error_2 \times x_2$: $(Error_2 = -3, x_2 = 2)$ -3*2 = -6

θ_1_new = *0-0.1(-4)*
θ_1_new = *0.4*

Results:
Predictions: $\hat{y}_1 = 0$, $\hat{y}_2 = 0$
Errors: -2, -3
New values: $\theta_0 = 0.25$, $\theta_1 = 0.4$

Iteration 2
$\theta_0 = 0.25$
$\theta_1 = 0.4$
$\alpha = 0.1$
Point 1: $(x_1, y_1) = (1, 2)$
Point 2: $(x_2, y_2) = (2, 3)$

Prediction formula: $\hat{y} = \theta_0 + \theta_1 \cdot x$

$\hat{y}_1 = 0.25+0.4 * 1 = 0.65$
$\hat{y}_2 = 0.25+0.4 * 2 = 1.05$

Error formula: $\hat{y} - y$ (predicted – actual)

Error1 for Point1: $(\hat{y}_1 = 0, y_1 = 2)$ 0.65-2=-1.35
Error2 for Point2: $(\hat{y}_2 = 0, y_2 = 3)$ 1.05-3 = -1.95

Update formula (intercept) is: θ_0_new = θ_0_old - $\alpha \cdot$ (average of errors)

θ_0_new **= 0.25-0.1((-1.35+-1.95)/2))**

θ_0_new **= 0.25-0.1(-1.65)**

θ_0_new **= 0.25- -0.165**

θ_0_new **= 0.25 + 0.165**

θ_0_new **= 0.415**

Update formula (slope) is: θ_1_new = θ_1_old - $\alpha \cdot$ (average of errors × x)

θ_1_new = θ_1_old - $\alpha \cdot$ (average of [Error × x])

$\text{Error}_1 \times x_1$: $(\text{Error}_1 = -1.35, x_1 = 1)$ -1.35*1= -1.35
$\text{Error}_2 \times x_2$: $(\text{Error}_2 = -1.95, x_2 = 2)$ -1.95*2 = -3.9

-1.35+-3.9=-5.25
-5.25/2= -2.625

θ_1_new = *0.4-0.1(-2.625)*

θ_1_new = *0.4- -0.2625*
θ_1_new = *0.4+0.2625*
θ_1_new = *0.6625*

Results:
Predictions: $\hat{y}_1 = 0.65$, $\hat{y}_2 = 1.05$
Errors: -1.35, -1.95
New values: $\theta_0 = 0.415$, $\theta_1 = 0.6625$

(4 pts) After two steps, are your predictions closer to the actual y-values? Why or why not? (A brief explanation is sufficient.)
YOUR ANSWER:

- Yes, they are closer to the actual y-values because the error values decreased. In the first iteration, the errors were -2 and -3, but after the second iteration they decreased to -1.35 and -1.95, showing that the predictions are improving and getting closer to the actual values.

Q3: Implementation & Bias Check (40 points)

1. Coding Simple Linear Regression (20 points)

a) Generate Data: Generate a synthetic dataset: for example, pick x-values from 0 to 10 (with about 20 points) and let y = 3x + 7 + (small random noise).

YOUR ANSWER / CODE NOTES:

- Files attached separately.

```python
# PART A: Generate Dataset
X= np.linspace(0, 10, num=20)
print("X:")
print(X)


noise = np.random.normal(0, 1, size=X.shape)  # small random noise
y = 3 * X + 7 + noise
print("y")
print(y)
```

```
X:
[ 0.          0.52631579  1.05263158  1.57894737  2.10526316  2.63157895
  3.15789474  3.68421053  4.21052632  4.73684211  5.26315789  5.78947368
  6.31578947  6.84210526  7.36842105  7.89473684  8.42105263  8.94736842
  9.47368421 10.        ]
y
[ 6.54023992  8.32906062 10.36690002 11.60413233 15.0539592  14.22037068
 16.53742457 17.52964182 18.98556155 19.74065766 23.26006966 24.17203825
 26.21336811 27.29545199 30.7170375  29.79904447 31.66138167 35.17551522
 33.41813623 37.19314872]
```

b) Split: Split your data into 80% Training and 20% Testing.

YOUR ANSWER / CODE NOTES:

```python
# PART B: Split into train and test sets for demonstration (80/20 split
X = X.reshape(-1, 1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2

print("Training X:", X_train.shape)
print("Testing X:", X_test.shape)
```

```
Training X: (16, 1)
Testing X: (4, 1)
```

c) Fit: Linear Regression. (For Python, you can use scikit-learn's LinearRegression to fit the model). Report the learned intercept and coefficient.

YOUR ANSWER / CODE NOTES:

Learned Intercept (θ_0): See below.

Learned Coefficient (θ_1): See below.

```python
# PART C: Report the learned intecept and coeffiecient
print("Intercept:", model_accuracy.intercept_)
print("Slope (coefficient):", model_accuracy.coef_[0])
```

```
Intercept: 7.255902560612615
Slope (coefficient): 2.9271477327195625
```

d) Evaluate: Calculate and print the Mean Squared Error (MSE) for both the training and test sets.

YOUR ANSWER / CODE NOTES:

Training MSE: See below.

Testing MSE: See below.

```python
# PART D: Show MSE for Train/Test sets
# MSE for Train
mse_train = mean_squared_error(y_train, model_accuracy.predict(X_train)
print("Training MSE:", mse_train)


# MSE for Test
y_pred = model_accuracy.predict(X_test)   # predictions for test data
mse = mean_squared_error(y_test, y_pred)  # calculate MSE

print("Test MSE:", mse)
```

```
Training MSE: 0.8090452280407672
Test MSE: 1.010393021778099
```

2. The "Extrapolation Bias" Experiment (20 points)

a) Generate New Data: Generate 20 points where x is between 0 and 5 (Low Range), and generate y using the same formula as in (Q3-1a).

YOUR ANSWER / CODE NOTES:

```python
# PART A: Generate Dataset (LOW RANGE, training)
X_train= np.linspace(0, 5, num=20)
X_train = X_train.reshape(-1, 1)
print("X:")
print(X_train)


noise = np.random.normal(0, 1, size=X_train.shape)  # small random noise
y_train = 3 * X_train + 7 + noise
print("y")
print(y_train)
```

```
X:
[[0.        ]
 [0.26315789]
 [0.52631579]
 [0.78947368]
 [1.05263158]
 [1.31578947]
 [1.57894737]
 [1.84210526]
```

b) Train: Train a new Linear Regression model only on this "Low Range" data (0 to 5).

YOUR ANSWER / CODE NOTES:

[9]
0s

```python
print("Training Model...")
model1 = LinearRegression()
model1.fit(X_train, y_train)
y_pred = model1.predict(X_test)
print("Model trained.")
print("-" * 30)
```

```
Training Model...
Model trained.
------------------------------
```

c) Test on New Range: Generate a test set where x is between 10 and 15 (High Range). Use your model to predict these values.

```python
# PART A: Generate Dataset (HIGH RANGE, test)
X_test= np.linspace(10, 15, num=20)
X_test = X_test.reshape(-1, 1)
print("X:")
print(X_test)

# No need to generate y, cause that is what we are predicting.
```

```
X:
```

YOUR ANSWER / CODE NOTES:

d) Plot & Reflect:

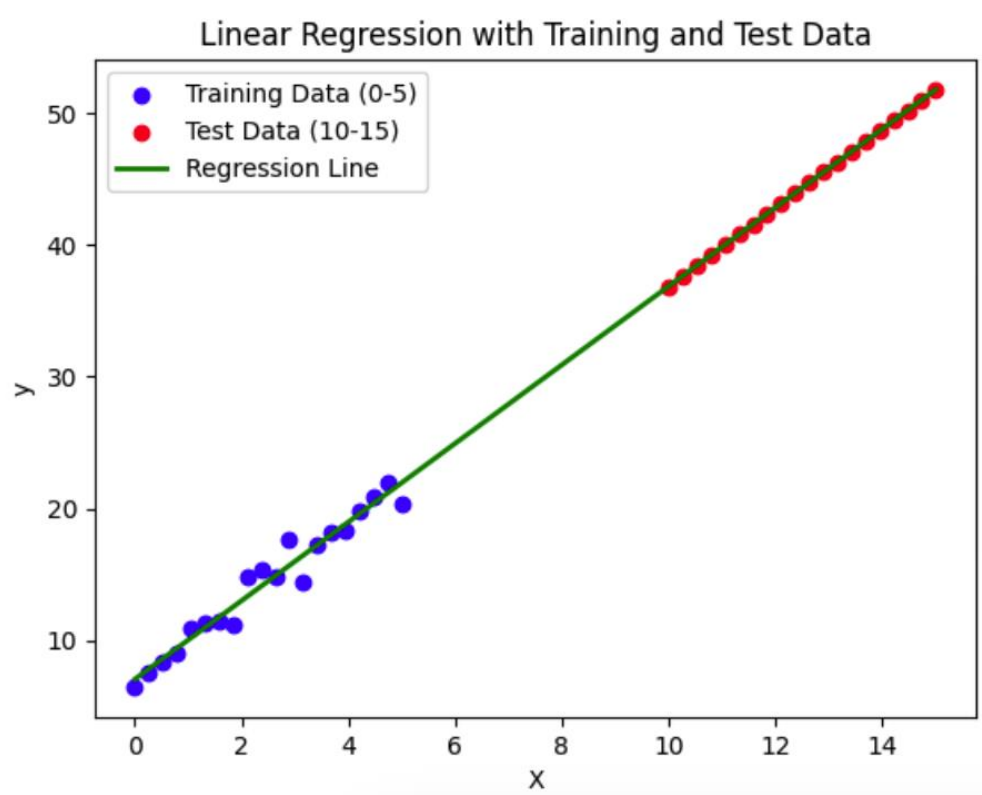
- Create a scatter plot showing the Training Data (0-5) in Blue and the Test Data (10-15) in Red.
    - See below.
- Draw your model's regression line extending through both ranges.

- See below.

- Question: Does the model predict the "High Range" well? What does this tell you about the risks of using Machine Learning on data that is different from what it was trained on? (This is a form of Sampling Bias).
  - The model predicts the high range well in this synthetic example because the relationship is linear and simple. However, in real-world problems, data is often noisy and not always linear. Therefore, predictions outside the training range can be risky and inaccurate at times. This helps to emphasize extrapolation bias, where models can fail when applied to data that differs from what they were trained on. In this case it was trained on the lower range, but tests utilized higher range data not accounted for in the training dataset.

```
plt.scatter(X_train, y_train, color='blue', label='Training Data (0-5)')
plt.scatter(X_test, y_pred, color='red', label='Test Data (10-15)')

x_line = np.linspace(0, 15, 100).reshape(-1, 1)
y_line = model1.predict(x_line)
plt.plot(x_line, y_line, color='green', linewidth=2, label='Regression Line')
plt.xlabel("X")
plt.ylabel("y")
plt.title("Linear Regression with Training and Test Data")
plt.legend()
plt.show()
```



Q4: The "Biased History" Experiment (15 points)

In class, we discussed how Machine Learning models can "parrot" historical biases found in data (e.g., Gender Bias in Hiring). In this coding task, you will create a synthetic dataset to prove how easily a Linear Regression model learns to discriminate.

1. Generate "Biased" Data (5 points)

Create a simple synthetic dataset representing employees at a company.

- Experience: Create a list of 10 employees with years of experience: [1, 2, 3, 4, 5, 1, 2, 3, 4, 5]
- Group: Create a list representing two groups (0 and 1): [0, 0, 0, 0, 0, 1, 1, 1, 1, 1]
- Salary: Generate the salaries using this formula:

• Base Salary = Experience * 2000
• The Bias: Add $10,000 extra to everyone in Group 1. (Group 0 gets $0 bonus).
• Example: An employee with 2 years' experience in Group 0 gets $4,000. In Group 1, they get $14,000.

YOUR ANSWER / CODE NOTES:

```python
# Create a simple synthetic dataset representing company employees
experience = [1,2,3,4,5,1,2,3,4,5]
group = [0,0,0,0,0,1,1,1,1,1]

salary = []
for exp, grp in zip(experience, group):
  base = exp * 2000
  if grp == 1:
    base = base + 10000
  salary.append(base)

print("Experience:")
print(experience)

print("Group:")
print(group)

print("Salary:")
print(salary)
```

```
Experience:
[1, 2, 3, 4, 5, 1, 2, 3, 4, 5]
Group:
[0, 0, 0, 0, 0, 1, 1, 1, 1, 1]
Salary:
[2000, 4000, 6000, 8000, 10000, 12000, 14000, 16000, 18000, 20000]
```

2. Train the Model (5 points)

- Combine "Experience" and "Group" into your feature matrix (X) (it should have 2 columns).
- Fit a LinearRegression model to predict "Salary" (y).

YOUR ANSWER / CODE NOTES:

```
# Train the Model
X = np.column_stack((experience, group))
print(X)
```

```
[[1 0]
 [2 0]
 [3 0]
 [4 0]
 [5 0]
 [1 1]
 [2 1]
 [3 1]
 [4 1]
 [5 1]]
```

```
print("Training Model...")
model = LinearRegression()
model.fit(X, salary)
y_pred = model.predict(X)
print("Model trained.")
print("-" * 30)
```

```
Training Model...
Model trained.
------------------------------
```

```
y_pred # salary prediction
```

```
array([ 2000.,  4000.,  6000.,  8000., 10000., 12000., 14000., 16000.,
       18000., 20000.])
```

3. The Fairness Test (5 points)

- Imagine two new job applicants who have the exact same experience level (e.g., 6 years).

• Applicant A: 6 Years Experience, Group 0.
• Applicant B: 6 Years Experience, Group 1.

- Use your model to predict the salary for both.

YOUR ANSWER:

Predicted Salary for Applicant A: ***$12000***
Predicted Salary for Applicant B: ***$22,000***

- Reflection: Does the model give them the same salary? Why not? Explain in 1-2 sentences why automating the decision process did not fix the bias.

YOUR ANSWER: _____

```
print(y_pred) # salary prediction
print("Six Years Included Prediction:")
print(y_pred_6) # salary prediction w/ 6exp
```

```
[ 2000.  4000.  6000.  8000. 10000. 12000. 14000. 16000. 18000. 20000.]
Six Years Included Prediction:
[ 2000.  4000.  6000.  8000. 10000. 12000. 12000. 14000. 16000. 18000.
 20000. 22000.]
```

- No. The model does not give them the same salary. The model is now biased because it learned patterns from historical data that include group-based differences between group 0 and group 1. Therefore, automating the decision process alone does not remove the underlying bias in the data.